Every product correctly rounded: bit-reproducible LoRA training of a 375-billion-parameter mixture of experts from a laptop's disk

Shaban Ejupi
Independent researcher

31 October 2026

Abstract

Floating-point training is not a function of its inputs. GPU kernels choose their reduction order by shape, backward passes accumulate with atomics, and the same data in another batch, on another GPU or with another library gives other gradients. Recent batch-invariant and deterministic kernels fix one order per kernel: a run becomes repeatable on the same software and hardware, but its result is still defined by an implementation. We take the other route: every linear map of the forward and of the backward pass returns the *correctly rounded* value of its exact sum, a number defined by the mathematics alone. The products run exactly on INT8 tensor cores from digit planes of BF16 and FP8 operands; the rounding is certified, and every output the certificate leaves open is recomputed exactly on the CUDA cores or, as a last resort, in a 2560-bit host superaccumulator. Gradients through FP8 expert weights are exact, never quantized. Applied to rank-16 LoRA on Jais-3-375B-A23B-FP8 (61 layers, 192 FP8 experts, a 391 GB checkpoint) streamed from the NVMe disk of a laptop with a 12 GB GPU once per forward and once per backward, a step of 8×1024 tokens takes about 30 minutes (1 792–2 009 s). The step does not depend on how rows are grouped into products (4 096 or 40 rows), on the number of threads (24 or 7), on interruption (a resumed run is byte-identical) or on which of three generations of our product kernels computes it; no output is ever used uncertified. Five hand-written backward formulas agree with central differences of their FP64 forward to 5.1×10^{-7}, and finite differences through the model and the predicted loss change of each update agree with the backward. Trained on 191.9 million tokens of public documents of Kosovo's institutions, the held-out loss falls from 1.5102 at step 1 to 1.463–1.468 at steps 15–18.

Përmbledhje (shqip). Trajnimi me numra me presje lëvizëse nuk është funksion i hyrjeve të tij: e njëjta e dhënë në një grup tjetër, në një GPU tjetër ose me një librari tjetër jep gradientë të tjerë. Këtu çdo shumëzim matricash i trajnimit, përpara dhe prapa, jep vlerën e *rrumbullakosur saktë* të shumës së saktë—një numër i përcaktuar vetëm nga matematika—i llogaritur mbi tensor core-t INT8, i certifikuar, dhe i rillogaritur saktë kur certifikata s'mjafton. Me këtë trajnuam LoRA me rang 16 mbi Jais-3-375B-A23B-FP8 (61 shtresa, 192 ekspertë FP8, 391 GB) nga disku NVMe i një laptopi me GPU 12 GB: një hap me 8×1024 tokena zgjat rreth 30 minuta, dhe nuk varet nga grupimi i rreshtave, numri i fijeve, ndërprerja apo versioni i kernel-ave; asnjë dalje e pacertifikuar nuk përdoret kurrë. Mbi 191,9 milionë tokena dokumentesh publike të institucioneve të Kosovës, humbja e mbajtur jashtë bie nga 1,5102 në 1,463–1,468.

1 Introduction

Two runs of the same training script on the same data rarely produce the same model. Floating-point addition is not associative, and a GPU kernel's order of summation depends on the shape of the problem (how K is split across thread blocks), on the library version and on the hardware; backward passes add partial gradients with atomic instructions whose order is decided at run

time. Zhuang et al. measured the consequence: nondeterministic tooling barely moves aggregate accuracy but makes performance on subsets of the data much more variable, and enforcing determinism cost up to 746% overhead [18].

The current answer is to fix the order. Batch-invariant kernels make a token's output independent of the batch it is computed in [10]; tree-based invariant kernels align the reduction order across tensor-parallel sizes [17]; fixed-configuration GEMMs give the same bits across three NVIDIA architectures for inference [4]; on-policy reinforcement learning with batch-invariant forward kernels and hand-written backward passes keeps trainer and sampler bitwise equal [16]; and DeepSeek-V4 replaces cuBLAS end to end, abandons split-K and replaces atomics in the attention and MoE backward by per-SM buffers and deterministic reductions, for training that is repeatable and aligned with inference [6]. All of these define the result by an implementation: the bits are those of one kernel with one tiling. Another kernel, another tile shape or another GPU generation gives other bits, and nobody outside can check a gradient without running the same code on the same kind of machine.

We define the result by the mathematics instead. Every linear map of the training step—every projection, every expert, the output layer, in the forward *and* the backward pass—returns the correctly rounded value of its exact sum. That value is unique, so it cannot depend on the order of summation, the tiling, the batch, the thread count or the machine; any third party can recompute any output with exact rational arithmetic and compare. INT8 tensor cores make this affordable: an INT8 product with INT32 accumulation is exact, and a BF16 or FP8 operand is a short sum of INT8 digits times powers of two, so the exact product is a sum of exact INT8 products [14, 9]. What remains is to know when the rounding is decided, and to finish the rare outputs where it is not.

The test case is deliberately out of proportion to the hardware. Jais-3-375B-A23B-FP8 (379 billion parameters, Apache-2.0, 61 layers, 192 routed FP8 experts per MoE layer) does not fit in the memory of any single GPU; we fine-tune it with rank-16 LoRA adapters [11] on a laptop whose GPU (RTX PRO 3000 Blackwell Laptop, sm_120, 12 GB) holds one layer at a time, streaming the checkpoint from the laptop's NVMe disk twice per step. Our contributions:

- correctly rounded products for every operand format of a modern FP8 MoE in training: BF16 projections with BF16 or FP32 outputs, FP8 experts with block scales as the model prescribes (W8A8), and the transposed products of the backward, including an exact gradient through FP8 weights (Section 3);
- a complete LoRA training step of a 375B MoE from disk on one laptop GPU, with every host-side operation in a fixed FP64 order (Section 4);
- evidence that the step is a function of its inputs: byte-identical gradients across rows per product, thread counts, interruption and three generations of product kernels, and gradient checks (Section 5);
- measurements of what exactness costs and where the time goes, and a first training run on public data (Sections 6, 7).

To our knowledge, no earlier training run has had every product of its forward and backward pass correctly rounded, and no earlier LoRA fine-tuning of a model of this size has run on a single laptop GPU.

2 Background

LoRA. A frozen map $y = Wx$ gains a trainable low-rank term, $y = Wx + s\,BAx$ with $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{m\times r}$ and scale $s = \alpha/r$ [11]. We follow the PEFT library's defaults: $B = 0$ and A uniform in $\pm 1/\sqrt{k}$, so the first step's $\partial L/\partial A$ is exactly zero. QLoRA [8] quantizes the frozen weights to fit a GPU; offloading systems move parameters and optimizer state to CPU memory and NVMe

instead, and fine-tune trillion-parameter models on one DGX-2 node (16 GPUs) [15] or reduce the host memory that SSD offloading needs [13]. We keep the checkpoint's own formats and stream it, one layer at a time, through a single GPU.

Fine-grained FP8. Jais-3's experts store E4M3 weights with one BF16 scale per 128×128 block and quantize each token's activation per group of 128 with an FP32 scale before the product (the W8A8 scheme DeepSeek-V3 introduced at scale [5]). Attention projections, the shared expert, the router and the output layer are BF16.

Exact products on integer units. The Ozaki scheme splits floating-point matrices into integer slices whose products integer tensor cores compute exactly [14]; our companion paper builds FP64, multi-word and arbitrary precision on the INT8 units of the same laptop GPU and correctly rounded MXFP4, NVFP4 and BF16 products for inference of the largest open models [9]. Reproducible and correctly rounded summation on CPUs and GPUs has a long history—long accumulators [12], reproducible BLAS [7, 2] and accurately rounded BLAS [3]—but not, as far as we know, inside a training loop.

3 Correctly rounded products for training

BF16 × BF16. For $Y = XW^T$ each block of 128 consecutive k of a weight row or a token row shares a grid at its largest exponent; on that grid each element is an integer, cut into signed 7-bit digits. Weights take 3 digits and tokens 4 when the output is BF16 ($p = 8$), 5 and 6 when it is FP32 ($p = 24$). Each pair of digit planes is one INT8 GEMM with INT32 accumulation (`mma.m16n8k32`), and the partial sums of a block are combined in an `int64` window at the token's largest block, with headroom for the number of blocks. Elements below the grid are floored; their count bounds the tail, and the output is certified when the whole interval rounds to one floating-point value (round to nearest even, subnormals included). Our tiled kernel computes 64 rows × 32 tokens per block of 256 threads, streaming K through 3–4 `cp.async` stages and loading fragments with `ldmatrix`.

The fixup. An uncertified output is written as NaN and recomputed exactly on the CUDA cores, one warp per output: every product is formed exactly and summed in a 128-bit fixed point anchored at the largest product of that row and token. If even that window cannot certify it, the output is finished on the host in a 2 560-bit superaccumulator with no window and no tail, which always succeeds. Nothing uncertified is ever used.

FP8 experts, forward. The activation is first quantized exactly as the model prescribes (E4M3 per group of 128 with an FP32 scale); the product of E4M3 weights and E4M3 activations is then exact in three digits each, and the block term $(S\,s)\sum wx$ fits a $64 \times 32 \to 128$-bit product shifted into the 63-bit window. Outputs the window leaves open are recomputed exactly as the reference FP8 kernel does, so every output—NaN or not—equals that kernel's.

The backward through FP8 weights, exactly. The gradient of an input is $dX = dY\,W$. We round dY once to BF16, the gradient format of BF16 training, and never to FP8. An E4M3 weight (4 significant bits) times its BF16 block scale (8 bits) is exact in FP32 and splits exactly into two BF16 numbers $w_{hi} + w_{lo}$, so $[\,dY \,|\, dY\,]\,[W_{hi}; W_{lo}]$ is the exact product with the FP8 weights. Our current kernel avoids the doubling: it transposes the weight's own E4M3 digits (three, exact) and applies the block scale per block of 128, so a term costs $3 \times 6 = 18$ digit products instead of $2 \times 4 \times 6 = 48$, with the same correctly rounded outputs. Maps that share an input form one product (q, k, v together; an expert's gate and up together), rounded once.

LoRA gradients. With $G = \partial L/\partial$(projection output) before RoPE and $H = \mathrm{BF16}(xA^T)$, $\partial L/\partial B = s\,G^T H$ is one certified product over the tokens (K = number of tokens). No bit of it can depend on the order of the tokens or on how rows were cut into products.

4 A training step of a 375B MoE on a laptop

Forward. The model runs layer by layer over all rows of the step (8 training sequences of 1 024 tokens and 2 held-out sequences), each layer read from disk once and applied to every row; the residual stream entering each layer is kept in host memory (activation checkpointing at layer granularity [1]). Each expert is read once per layer and applied to all rows routed to it, in ascending expert id, and each row's weighted sum of its experts is accumulated in FP64 in that order and rounded once. Run this way, the batched model gives the token-by-token model bit for bit [9].

Backward. From the top, every layer is read from disk once more and recomputed from its saved input with the forward's own functions; then every map's dX is one correctly rounded product (FP32 outputs) added to an FP64 gradient. Attention runs in FP64 with every sum in a fixed order: the scores, the probabilities' gradient, dq, dk and dv on the GPU's FP64 units with `__dmul_rn`/`__dadd_rn` in exactly the host's order, the softmax with the host's `exp`. RoPE's inverse rotation, RMSNorm and SwiGLU are FP64 on the host. The routing weights $w_e = 2.5\,s_e/\sum s$ (sigmoids of the eight chosen experts) pass their gradient through the sigmoid to the router logits and its transposed product; the top-8 choice is a constant, as in every MoE backward. The backward reads only the experts the training rows were routed to: 376–379 GB per step.

Streaming. A reader thread fetches the tensors in the order they will be used into a pinned ring; the time spent waiting for it counts as disk time, and in the backward it is 0.0 s per step: the disk keeps ahead of the computation. The host's peak resident memory is 25.9–26.9 GB.

Optimizer and checkpoints. Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, no weight decay, learning rate 10^{-4}) updates B from step 1 and A from step 2. After every step the adapters and both moments are written to a temporary file and renamed; a run of n steps continues from the last checkpoint.

5 Verification

Invariance. Table 1 collects the tests. The third row is the one fixed-order kernels cannot pass: the three kernel generations differ in tile shapes, number of digits, digit products per term (48 against 18 for the experts' gradient) and epilogue width (125 against 63 bits), and still give the same bits, because each output is the unique correctly rounded value. The 63-bit window leaves about 0.02% of the outputs to the fixup where the 125-bit one left none: the same outputs from a third of the epilogue's instructions.

Gradients. Each hand-written backward formula—attention, RoPE, RMSNorm, SwiGLU and the MoE gating—agrees with central differences of its own FP64 forward on a 2^{-12} grid to a relative 5.1×10^{-7}, and a deliberately wrong formula fails. Through the 4-layer model (three dense layers and one MoE layer, 128 tokens), moving the embeddings in 5 random directions by $\epsilon = 0.004$ changes the loss with slope 0.96 against $\langle g_0, \Delta\rangle$; the residual is the BF16 rounding noise of the forward ($\sim 2 \times 10^{-3}$) and grows with ϵ. On real updates (1 024 tokens), the loss changed by -0.04588 and -0.02731 where the first-order prediction was -0.04135 and -0.03288.

Table 1: The step is a function of its inputs.

test	result
step 1 of the 61-layer model, 1 024 tokens: 4 096 vs 40 rows per product, 24 vs 7 threads	122 dB files byte-identical; dB, dA and g_0 digests identical
2 steps in one run vs 1 + 1 resumed from the checkpoint (4 layers)	checkpoints byte-identical
three generations of product kernels (`bf16_gemm2` vs tiled; $[dY\|dY][W_{hi};W_{lo}]$ vs E4M3 digits; reference FP8 kernel vs tiled), 4 layers	digests `63e9904ca4c9d00f` (128 tokens × 2) and `1dd8193b18cc200b` ($8 \times 1024 + 2$ held out) unchanged
experts' dX, old vs new kernel (layer 3, expert 0, 341 tokens)	0 outputs differ; 4 096 checked against the superaccumulator agree bit for bit
attention backward, GPU vs host (6, 45, 131, 1 024 tokens)	0 (2–7 a step before the host superaccumulator was added)
outputs used uncertified	

Table 2: Training steps of the 61-layer model, 8×1024 tokens (+ 2 held-out sequences in the forward), from the training log; "MoE bwd" is the MLP and MoE backward, "attention bwd" the attention and LoRA backward.

kernels (step)	step [s]	forward [s]	MoE bwd [s]	attention bwd [s]
digit planes, `bf16_gemm2`, GPU attention forward (5)*	3 507.9	1 192.6	1 505.8	793.1
+ GPU attention recomputed in the backward (7)*	2 764.3	721.6	1 448.9	577.1
+ tiled products, GPU attention backward (9–14)	1 967–2 009	634–639	884–895	462–468
same kernels, at night (16)	1 805.5			
+ tiled FP8 expert forward (17)	1 791.8			

* kernel tests shared the GPU with these steps.

6 Performance

A step now takes about 30 minutes (Table 2); the spread between 1 806 and 2 009 s with the same kernels is other work on the same laptop. Table 3 lists the kernel changes behind the fall from 3 508 s. One of them was not about arithmetic at all: digit planes written one byte per thread to scattered sectors took 27.5 of the 28 s of an MoE layer's backward until one warp per block wrote 4-byte words.

What exactness costs. An exact output needs 12 INT8 products per term for a BF16 result and 30 for an FP32 one. At 69–93 TOPS of digit products that is roughly 15–29 times slower than cuBLAS BF16 on the same GPU (84–91 TFLOPS measured). In the step this is not the bottleneck: the forward pass's products take 186 of its 524 s. The rest is streaming 391 GB through host memory twice, the FP64 host work (attention softmax, norms, SwiGLU) and per-expert overheads: 192 experts per layer are applied one after another, each with its own transfers.

7 Training on public data

The corpus was harvested from the public websites of Kosovo's institutions and other public sources, politely (`robots.txt`, at most one request per second per host). The largest shares come from the Central Bank of the Republic of Kosovo (40.8 million tokens), the Ministry of Justice (26.0 M), the Ministry of Industry, Entrepreneurship and Trade (19.3 M) and the Ministry of Education, Science, Technology and Innovation (13.5 M). Lines repeated in at least 20% of a

Table 3: Kernels, measured on the laptop while the training shared the GPU.

kernel	before → after
BF16 product, shared expert's gate, 4 096 tokens, $p = 8$ (TOPS of digit products)	26.1 → 15.7 ms (41 → 69)
same, $p = 24$	48.7 → 29.2 ms (56 → 93)
one expert's dX (down + gate/up), 341 tokens, host work included	81.7 → 12.2 ms
one expert's forward (gate, $p = 8$), 432 tokens (a step's mean)	1.79 → 1.30 ms
digit planes of an MoE layer's backward	27.5 → 0.7 s
attention backward, 1 024 tokens, host FP64 → GPU FP64	2.06 → 0.69 s

Table 4: Held-out loss (2 sequences, 2 046 predictions).

step	1	2	3	4	15	16	17	18
loss	1.51019	1.50738	1.50570	1.50056	1.46402	1.46337	1.46709	1.46759

source's documents, documents under 500 characters and exact duplicates are removed; Jais-3's tokenizer turns the rest into 187 437 sequences of 1 024 tokens (191.9 million tokens). Each step takes 8 sequences in a seeded order; the last 2 sequences of the corpus are held out and run through every forward.

The held-out loss falls by 0.047 in 16 steps (Table 4) and then holds within the variation of single 8-sequence batches; after 18 steps the model has seen 147 456 training tokens, under 0.08% of the corpus. We report this run to show that the exact step trains, not as a claim about the adapted model's quality.

8 Limitations

- *Speed.* Thirty minutes per 8 192 training tokens is slow; the step is a feasibility and reproducibility result on a laptop, not a throughput result. The per-expert overheads and the FP64 attention backward are the next targets.
- *What is fixed, and what is unique.* The products are unique; attention, norms and SwiGLU are reproducible because their FP64 operations run in a fixed order, and their transcendental functions come from the host's C library. Bit identity across machines therefore assumes the same `exp`; a correctly rounded library (e.g. CORE-MATH) would remove that assumption.
- *Modelling choices.* dY is rounded once to BF16 before each transposed product, as BF16 training does; the top-8 routing is treated as a constant.
- *Scale of the evaluation.* One model, one adapter configuration, 18 steps; no comparison of downstream quality.

9 Conclusion

Correct rounding turns a training step into a mathematical function of its inputs. That is a stronger property than determinism: it survives a change of batch, of thread count, of interruption and of kernel, and it can be checked by anyone with exact arithmetic, without our code or our hardware. INT8 tensor cores make the products cheap enough that, for a model streamed from disk, exactness is not what limits the step. We trained a 375-billion-parameter MoE this way on one laptop GPU, and every one of its products was certified.

Code and data. The code is source-available under the Ozaki-Blackwell Restricted Source License 1.0 at https://github.com/ShabanEjupi/ozaki-blackwell (files `llm/jais_lora.cu` and `llm/train.sh`); the training logs print FNV-1a digests of every dB and of the embedding gradient, so a run can be checked bit for bit.

References

[1] T. Chen, B. Xu, C. Zhang, C. Guestrin. Training deep nets with sublinear memory cost. arXiv:1604.06174, 2016.

[2] S. Collange, D. Defour, S. Graillat, R. Iakymchuk. Numerical reproducibility for the parallel reduction on multi- and many-core architectures. *Parallel Computing* 49:83–97, 2015.

[3] C. Chohra, P. Langlois, D. Parello. Reproducible, accurately rounded and efficient BLAS. Euro-Par 2016 Workshops, LNCS 10104, 2017.

[4] L. Cooper, S. Jeong, H. Jeon, J. Young, H. Kim. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. arXiv:2609.25624, 2026.

[5] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.

[6] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. arXiv:2606.19348, 2026, Section 3.3.

[7] J. Demmel, H. D. Nguyen. Parallel reproducible summation. *IEEE Trans. Computers* 64(7):2060–2070, 2015.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. arXiv:2305.14314, 2023.

[9] S. Ejupi. Precision as a software choice: FP64, double-double, triple-double and beyond from the INT8 tensor cores of a laptop GPU. Preprint, 29 October 2026.

[10] H. He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Blog post, 10 September 2025, https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.

[12] U. Kulisch. *Computer Arithmetic and Validity: Theory, Implementation, and Applications*, 2nd ed. De Gruyter, 2013.

[13] Y.-C. Liaw, S.-H. Chen. MemAscend: System memory optimization for SSD-offloaded LLM finetuning. arXiv:2505.23254, 2025.

[14] K. Ozaki, Y. Uchino, T. Imamura. Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique. arXiv:2504.08009, 2025.

[15] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, Y. He. ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning. arXiv:2104.07857, 2021.

[16] B. Wasti, W. Ye, T. Rao, M. Goin, P. Zhang, T. Liu, N. Gimelshein, W. Kwon, K. You, Z. Li. No more train-inference mismatch: bitwise consistent on-policy reinforcement learning with vLLM and TorchTitan. vLLM blog, 10 November 2025, https://vllm.ai/blog/bitwise-consistent-train-inference.

[17] Z. Zhang, X. Ding, J. Yuan, R. Liu, H. Mao, J. Xing, Z. Liu. Deterministic inference across tensor parallel sizes that eliminates training-inference mismatch. arXiv:2511.17826, 2025.

[18] D. Zhuang, X. Zhang, S. L. Song, S. Hooker. Randomness in neural network training: characterizing the impact of tooling. arXiv:2106.11872, 2021.